

Random Forest con scikit-learn

Clasificación de pasajeros del dataset Spaceship Titanic

Entregable:	Uso de framework o biblioteca de aprendizaje máquina para la implementación de una solución
Materia:	Inteligencia artificial avanzada para la ciencia de datos
Alumno:	Yael Michel García López
Matricula:	A01750911
Fecha:	09/09/2026
Repositorio:	https://github.com/yaelgl/Momento_Retroalimentacion_A01750911

Resultado principal

El modelo alcanza una exactitud de 80.06% sobre 2608 pasajeros que nunca vio durante el entrenamiento, contra 50.35% de un clasificador trivial. La validación cruzada de 5 pliegues da 80.07% con una desviación de solo 0.0038, prácticamente igual que la medición en prueba.

1. Objetivo y algoritmo elegido

El objetivo es implementar un algoritmo de aprendizaje maquina usando un framework, demostrando que se sabe configurarlo, y comprobar que hace predicciones sobre datos nuevos.

El framework es scikit-learn y el algoritmo elegido es Random Forest, un conjunto de arboles de decision que votan. Se eligio por dos razones. La primera es que el entregable pide justificar con matriz de confusion, y eso requiere un problema supervisado con una respuesta correcta contra la cual comparar; los metodos de agrupamiento no encajan ahi. La segunda es que en la Parte I de este modulo se programo a mano un arbol de decision, asi que usar un bosque permite comparar directamente las dos partes sobre el mismo dataset.

2. Datos de entrenamiento y de prueba

Se usa el dataset Spaceship Titanic. Cada fila es un pasajero y la variable a predecir es Transported: si fue transportado a otra dimension o no, o sea clasificacion binaria.

Kaggle reparte dos archivos, pero test.csv no sirve para evaluar porque no incluye la columna Transported: las respuestas se quedan en el servidor de la competencia. Por eso la evaluacion se hace partiendo train.csv, el unico archivo con etiquetas, con train_test_split de scikit-learn usando stratify=y para conservar la proporcion de clases y random_state= 42 para que sea reproducible.

Conjunto	Pasajeros	Transportados	Proporcion	Uso
Entrenamiento	6085	3065	50.37%	Entrenar y ajustar
Prueba	2608	1313	50.35%	Medir una sola vez
Total (train.csv)	8693	4378	50.36%	-

Las clases estan balanceadas casi al 50%, dato que importa en la seccion 5 para justificar la metrica. Ademas el test.csv de Kaggle se usa al final para generar predicciones sobre datos completamente nuevos.

3. Preprocesamiento dentro de un Pipeline

Todo el preprocesamiento vive dentro de un Pipeline de scikit-learn, no en pasos sueltos antes de entrenar. El modelo completo tiene 3 pasos:

```
Pipeline([
    ("derivar", DerivarVariables()),      # transformador propio
    ("preprocesar", ColumnTransformer(...)), # imputar + codificar
    ("clasificador", RandomForestClassifier(...))
])
```

Hacerlo asi no es un detalle de estilo, evita un error concreto. Si uno imputa y codifica sobre el dataset completo antes de partirlo, los estadisticos del relleno se calculan mezclando entrenamiento y prueba, y el modelo termina usando informacion que no deberia conocer. Eso es fuga de informacion y hace que las metricas salgan mejores de lo que son. Dentro de un Pipeline, scikit-learn reajusta el imputador y el codificador cada vez que se llama a fit, y eso vale tambien dentro de cada pliegue de la validacion cruzada y de cada combinacion de la busqueda de hiperparametros.

Los tres pasos

DerivarVariables es un transformador escrito para este proyecto, heredando de BaseEstimator y TransformerMixin para que el framework lo trate como cualquier otro paso. Parte Cabin (que viene como "B/O/P") en cubierta y lado, saca el grupo de viaje de PassengerId para calcular TamGrupo y ViajaSolo, suma los cinco consumos en GastoTotal y marca a los menores de 12 anios.

ColumnTransformer aplica un tratamiento distinto a cada tipo de columna. A las numericas les imputa la mediana, no la media, porque los gastos estan muy sesgados a la derecha y la media quedaria arrastrada

por la cola. A las categoricas les asigna la categoria "Desconocido" en lugar de la moda, porque en este dataset la ausencia puede ser informativa y asi el modelo puede aprender de ella, y despues las codifica con OneHotEncoder usando `handle_unknown="ignore"` para que una categoria nunca vista no rompa la prediccion.

El resultado son 35 columnas numericas listas para el bosque. No se escala nada a proposito: los arboles parten con umbrales del tipo "Age <= 27" y esa comparacion no cambia si la columna se multiplica por una constante, asi que escalar no aportaria nada.

4. El algoritmo y su configuracion

Un arbol de decision solo es inestable: cambiando un poco los datos de entrenamiento puede elegir divisiones distintas y quedar bastante diferente. Tiene varianza alta. El bosque ataca eso con dos fuentes de aleatoriedad. La primera es bagging: cada arbol se entrena con una muestra distinta sacada con reemplazo del conjunto original. La segunda es el submuestreo de variables: en cada division, cada arbol solo puede escoger entre un subconjunto aleatorio de las columnas. Los arboles quedan distintos entre si, sus errores tienden a cancelarse al promediar los votos, y por eso el bosque generaliza mejor que un arbol solo.

Como se eligieron los hiperparametros

Con GridSearchCV, que prueba todas las combinaciones de una rejilla y evalua cada una con validacion cruzada estratificada de 5 pliegues. Fueron 48 combinaciones, o sea 240 ajustes del modelo.

El punto importante del procedimiento es que la busqueda recibe unicamente el conjunto de entrenamiento. El de prueba no participa y se mide una sola vez al final, con los parametros ya fijados. Si se eligieran los parametros mirando la prueba, esa medicion dejaria de ser honesta porque estaria escogiendo el modelo que mejor le queda justamente a esos datos.

Hiperparametro	Valor	Que controla
n_estimators	200	Cuantos arboles tiene el bosque
max_depth	14	Profundidad maxima de cada arbol
min_samples_leaf	5	Minimo de muestras por hoja
max_features	sqrt	Variables candidatas en cada division

`max_features` es el parametro mas caracteristico del bosque: es el que hace que los arboles sean distintos entre si. Con "sqrt" cada division considera la raiz cuadrada del total de columnas, unas 6 de 35. Si se pusiera None, todos los arboles verian todas las variables, quedarian muy parecidos y se perderia la ventaja de promediar. Sobre `n_estimators`, en la busqueda 400 arboles empataron con 200, asi que se conservo el modelo mas chico.

5. Metrica elegida y justificacion

La metrica principal es la exactitud (accuracy)

Es la proporcion de predicciones correctas sobre el total. Se eligio por tres razones concretas de este problema.

Primero, las clases estan balanceadas: 50.37% de los pasajeros fueron transportados. Ese es el punto clave, porque la critica habitual contra la exactitud es que engañe con datos desbalanceados: si el 95% de los casos fueran de una clase, predecir siempre esa clase daria 95% de exactitud sin haber aprendido nada. Aqui ese problema no existe, y de hecho el modelo trivial que siempre predice la clase mas frecuente sirve como piso de comparacion en 50.35%.

Segundo, los dos tipos de error cuestan lo mismo. No hay ninguna razon en este problema para que equivocarse diciendo "si fue transportado" sea peor que equivocarse al revés. Cuando los costos son

asimetricos, como en un diagnostico medico donde un falso negativo puede ser grave, conviene priorizar sensibilidad o precision. Aqui no aplica.

Tercero, es directamente interpretable y comparable: "acierta 8 de cada 10" se entiende sin explicacion, y permite comparar de frente contra el arbol programado a mano en la Parte I del modulo.

Por que no se eligio F1 como principal

El F1 combina precision y sensibilidad, que son dos metricas centradas en la clase positiva, y por construccion ignora los verdaderos negativos. En este problema acertar quien NO fue transportado importa exactamente igual que acertar quien si, asi que usar F1 como criterio principal dejaria fuera la mitad del problema. Se reporta de todos modos, pero como secundaria.

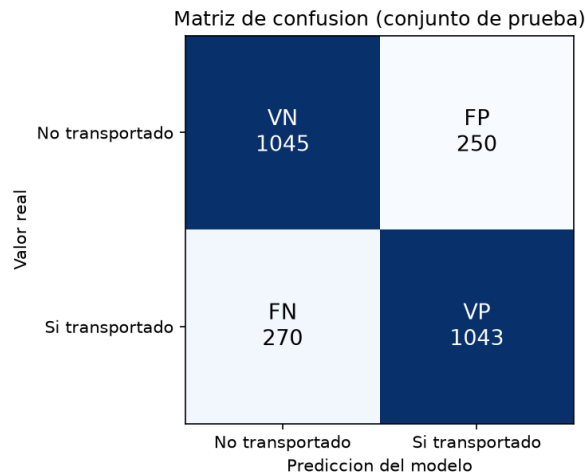
Metricas secundarias y para que sirve cada una

Precision, sensibilidad y especificidad sirven para verificar que el modelo no este inclinado hacia una clase. Reportar sensibilidad y especificidad juntas es justo lo que permite ver eso: si una fuera mucho mas alta que la otra, la exactitud global podria estar escondiendo que el modelo falla sistematicamente en un lado.

El ROC-AUC se incluye porque mide la capacidad de ordenar los casos por probabilidad sin depender del umbral de 0.5. El bosque no solo predice una clase, tambien entrega una probabilidad, y el AUC aprovecha esa informacion que la exactitud descarta.

6. Resultados

La matriz cruza las 2608 predicciones del conjunto de prueba contra lo que realmente paso. La clase positiva es "si fue transportado".

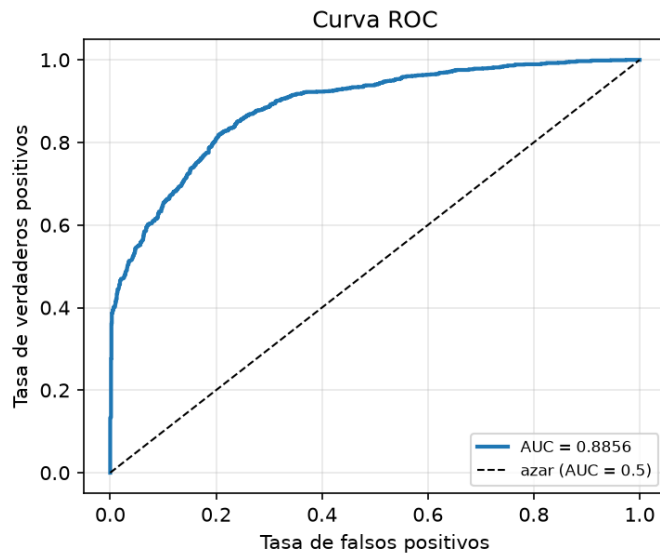


Los cuatro conteos son 1045 verdaderos negativos, 1043 verdaderos positivos, 250 falsos positivos y 270 falsos negativos. El modelo acierta 2088 de 2608 casos, y los dos tipos de error quedan muy parejos (250 contra 270), lo que confirma que no esta inclinado hacia ninguna clase.

Metrica	Formula	Prueba	Entrenamiento
Exactitud (principal)	$(VP+VN)/total$	80.06%	84.95%
Precision	$VP/(VP+FP)$	80.67%	84.97%
Sensibilidad	$VP/(VP+FN)$	79.44%	85.19%
Especificidad	$VN/(VN+FP)$	80.69%	84.70%
F1	media armonica	80.05%	85.08%
ROC-AUC	area bajo la curva	0.8856	0.9375

Sensibilidad 79.44% y especificidad 80.69% difieren apenas 1.26 puntos, asi que el modelo funciona practicamente igual de bien en las dos clases. El ROC-AUC de 0.8856 indica que ordena bien los casos

por probabilidad, bastante por encima del 0.5 que daría el azar.



7. Evidencia de que el modelo funciona correctamente

7.1 Validacion cruzada

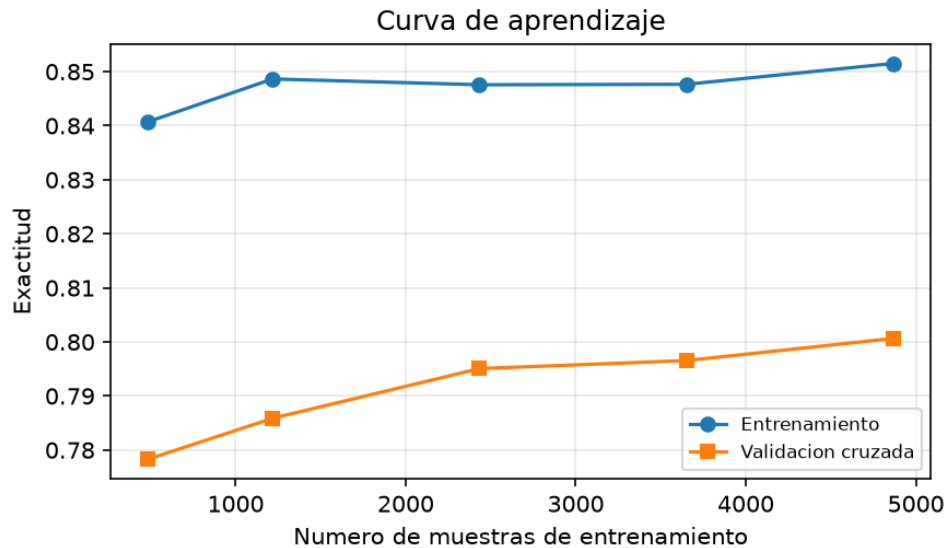
Una sola particion puede salir favorable por suerte. La validacion cruzada estratificada de 5 pliegues entrena y evalua cinco veces con reparticiones distintas: 0.8003, 0.8044, 0.8012, 0.8036, 0.7938. El promedio es 80.07% con desviacion estandar de 0.0038.

Ese numero es la mejor evidencia de todo el reporte, por dos motivos. La desviacion tan baja significa que el resultado casi no depende de como se partieron los datos, o sea que el modelo es estable. Y el promedio (80.07%) coincide con la medicion en el conjunto de prueba (80.06%), lo que confirma que la exactitud reportada no salio de una particion afortunada.

7.2 Sobreajuste

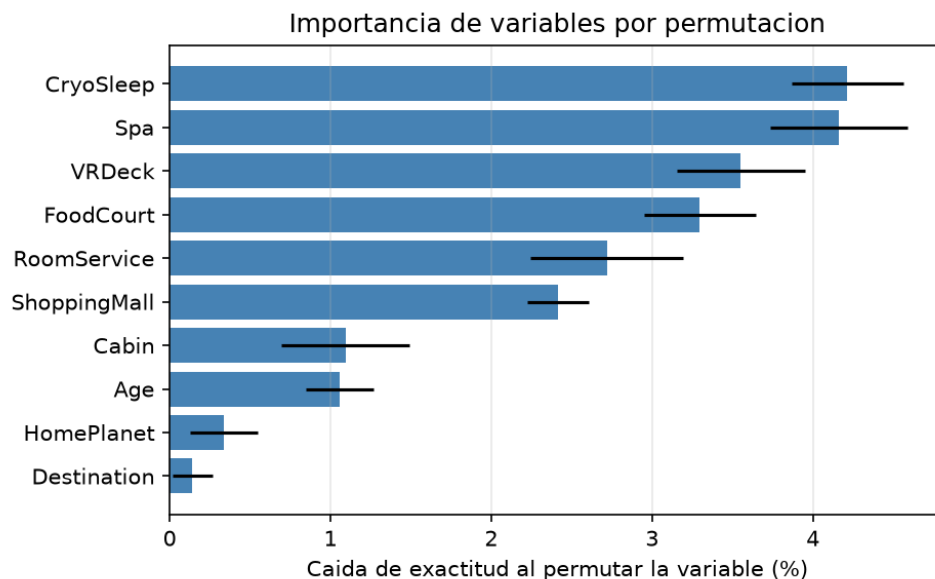
El modelo acierta 84.95% en entrenamiento y 80.06% en prueba, una brecha de 4.89 puntos. Que sea positiva es normal, al modelo siempre le va mejor con los datos que ya vio; lo importante es que sea chica. Una brecha de 20 o 30 puntos indicaria que memorizo el entrenamiento sin aprender nada generalizable.

La curva de aprendizaje muestra como cambia el desempenio segun cuantos datos se usan. La curva de validacion sigue subiendo levemente al llegar al 100%, lo que sugiere que mas datos ayudarian un poco, aunque ya se esta aplanando: el limite principal esta en el modelo, no en la cantidad de datos.



7.3 Importancia de variables

La importancia se midió por permutación: se revuelven al azar los valores de una columna y se mide cuanto empeora la exactitud. Se prefirió sobre la importancia por impureza que trae el bosque porque esa última tiende a inflar las variables numéricas continuas, simplemente porque ofrecen más puntos de corte posibles. La permutación mide el efecto real sobre el desempeño, y se calculó sobre el conjunto de prueba porque interesa saber que variables ayudan a generalizar.



Las tres variables más importantes son CryoSleep, Spa, VRDeck. El resultado tiene sentido: quienes viajaban en criosuenio no podían consumir nada, y son justamente los que se transportaron con mucha más frecuencia. Vale la pena notar el contraste con la Parte I, donde CryoSleep aparecía casi al final porque el árbol usaba GastoTotal en la raíz y eso volvía la pregunta redundante. Aquí la permutación evalúa cada variable dejando las demás intactas, y el bosque reparte el trabajo entre 200 árboles, así que la señal de CryoSleep sí se alcanza a ver.

8. Comparacion contra referencias

Modelo	Exactitud en prueba	Comentario
Trivial (clase mas frecuente)	50.35%	Piso de comparacion, no mira ninguna variable
Arbol de decision a mano (Parte I)	77.48%	Un solo arbol, sin framework
Random Forest (este trabajo)	80.06%	200 arboles con scikit-learn

El bosque supera al modelo trivial por 29.72 puntos porcentuales, que es la evidencia directa de que aprendio estructura real de los datos y no esta aprovechando el balance de las clases.

Frente al arbol programado a mano la mejora es de 2.58 puntos. Es una mejora moderada pero consistente con lo esperado: promediar 200 arboles reduce la varianza que un arbol solo no puede evitar. Conviene ser preciso con esta comparacion, porque las dos partes usan el mismo protocolo (70/30 estratificado con semilla 42) pero particiones distintas, ya que el repartidor de la Parte I estaba programado a mano y este usa `train_test_split`. Las filas exactas difieren, asi que la diferencia debe leerse como aproximada y no al segundo decimal.

9. El programa hace predicciones

Hay dos formas, las dos desde la terminal. El modo interactivo (`python main.py --interactivo`) pide los datos de un pasajero y devuelve la prediccion junto con la probabilidad, que es la proporcion de arboles que votaron a favor. Cada dato se valida antes de aceptarlo, asi que no admite una edad negativa ni una cubierta que no exista, y los campos que se dejen vacios los imputa el propio Pipeline. El modo por lotes (`python main.py --kaggle`) clasifica de una sola vez los 4277 pasajeros de `test.csv` y guarda el resultado con su probabilidad.

Sobre esas 4277 predicciones no se pueden calcular metricas porque no existen las etiquetas verdaderas. Lo que si se verifica es que la proporcion predicha se parezca a la del entrenamiento, y asi es: alrededor del 50%. Si el modelo estuviera sesgado hacia una clase, esa proporcion se iria a un extremo.

Aqui se nota una ventaja practica del Pipeline: al modo interactivo se le puede pasar una fila cruda, con nulos incluidos, y el propio Pipeline la deriva, la imputa y la codifica antes de clasificarla. No hay que repetir a mano el preprocesamiento del entrenamiento, que es donde suelen aparecer las inconsistencias entre lo que se entreno y lo que se predice.

10. Analisis y conclusiones

El Random Forest clasifica correctamente 80.06% de los pasajeros del conjunto de prueba, contra 50.35% del clasificador trivial, y la validacion cruzada confirma ese numero con una desviacion de 0.0038. El desempenio esta repartido de forma equilibrada entre las dos clases y la brecha entre entrenamiento y prueba es de 4.89 puntos, asi que el modelo generaliza en lugar de memorizar.

Limitaciones

El techo del dataset parece estar cerca del 80%. La curva de aprendizaje ya se esta aplanando, asi que juntar mas datos daria poco, y las competencias publicas sobre este dataset se mueven alrededor del 80 al 81% incluso con metodos mas pesados. Buena parte de lo que queda sin explicar probablemente sea ruido genuino.

El bosque tambien pierde algo que el arbol solo si tenia: la interpretabilidad directa. En la Parte I se podian leer las reglas aprendidas y seguir el camino de una prediccion; con 200 arboles votando eso ya no se puede, y hay que recurrir a herramientas indirectas como la importancia por permutacion. Es el intercambio habitual entre precision e interpretabilidad.

Que se podria mejorar

Lo mas prometedor seria probar metodos de boosting como HistGradientBoostingClassifier, que en tablas de este tipo suelen rendir un poco mas que el bosque. Tambien se podria buscar en una rejilla mas amplia con RandomizedSearchCV, que explora combinaciones al azar y cubre mas terreno con el mismo presupuesto de tiempo, y calibrar el umbral de decision si en algun momento importara mas un tipo de error que el otro.

Conclusion

Usar el framework cambio el trabajo de lugar. En la Parte I el esfuerzo estaba en programar correctamente el algoritmo; aqui el algoritmo son tres lineas, y el trabajo real esta en configurarlo y en montar el procedimiento de evaluacion sin equivocarse: meter el preprocesamiento dentro del Pipeline para no filtrar informacion, elegir los hiperparametros con validacion cruzada sin tocar el conjunto de prueba, y escoger una metrica que corresponda al problema en lugar de reportar la que salga mejor. El resultado, 80.06% con una desviacion de 0.0038 entre pliegues, es modesto frente al arbol hecho a mano, y eso tambien es informativo: buena parte del desempenio ya estaba en el planteamiento del problema y en la preparacion de los datos, no en la sofisticacion del modelo.